

# Spring Boot Hour 5 ❖❖❖ JPA, Hibernate & Spring Data JPA

## 🔊 Spring Boot — 8-Hour Plan · HOUR 5

### Talking to the Database: JPA, Hibernate & Spring Data JPA

*Goal: this hour untangles the three names that confuse every beginner — JPA, Hibernate, Spring Data JPA — then shows how your Java classes become database tables, and how Spring writes SQL from just a method name. Hours 5-6 are where most Spring Boot interview questions live. Take this one slow.*

#### 5.1 First, the problem: objects and tables speak different languages

Your Java world: **objects** — `User` with fields, references to other objects, inheritance. Your database world: **tables** — rows, columns, foreign keys. No objects, no references.

To save a `User`, someone must translate:

```
// Without any framework — raw JDBC. For ONE insert:
String sql = "INSERT INTO users (name, email) VALUES (?, ?)";
PreparedStatement ps = connection.prepareStatement(sql);
ps.setString(1, user.getName());
ps.setString(2, user.getEmail());
ps.executeUpdate();
// ...plus opening/closing connections, mapping ResultSet rows back
// to objects field by field, handling SQLExceptions... for EVERY query, EVERY table.
```

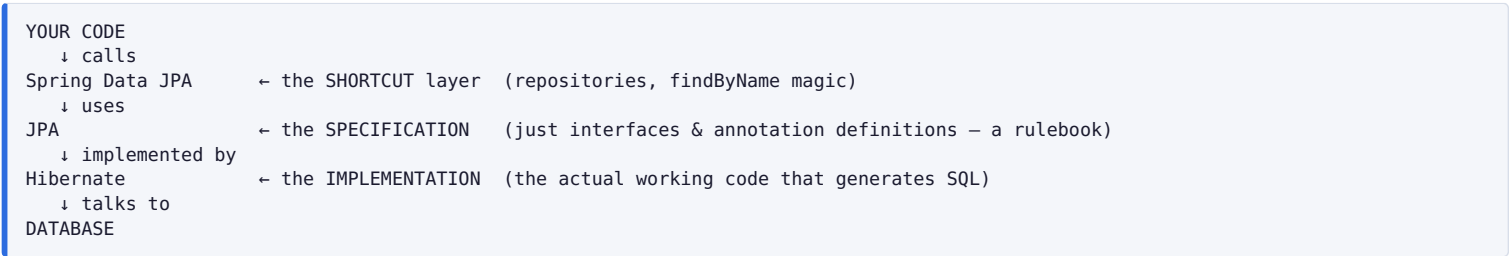
Multiply by 30 tables × 10 operations each. That's hundreds of lines of boring, error-prone translation code. This translation problem has a name — the **object-relational impedance mismatch** — and the solution category is **ORM: Object-Relational Mapping**. An ORM's job: *you work with objects; it generates the SQL*.

```
userRepository.save(user); // the same insert. That's it. The ORM wrote the SQL.
```

Analogy: you (Java objects) need to talk daily with a partner who speaks only SQL. Option 1: personally translate every sentence, both directions, forever (JDBC). Option 2: hire a professional interpreter who sits between you (ORM). You speak object; the DB hears SQL.

#### 5.2 The three names, untangled forever (top interview question of this hour)

Here's the layer cake, top to bottom:



**JPA (Jakarta Persistence API)** is a **specification** — a rulebook, not software. It *defines* what `@Entity`, `@Id`, `EntityManager` should mean, but contains no working code. You cannot "run JPA."

**Hibernate** is an **implementation** of that rulebook — real, working code that reads your annotations and generates actual SQL. It existed before JPA; it's the default implementation Spring Boot ships. (Others exist: EclipseLink, OpenJPA.)

**Spring Data JPA** is a **convenience layer on top** — it's what gives you `JpaRepository`, so instead of writing `EntityManager` code yourself, you declare an interface and Spring generates the data-access code.

Analogy: JPA is the syllabus ("every driving school must teach parking, signals, highway driving"). Hibernate is a driving school that actually teaches it — instructors, cars, the real work. Spring Data JPA is a premium concierge: you say "make my cousin a licensed driver," and it handles enrolling, scheduling, paperwork with the school. Syllabus → school → concierge.

**Why have a specification at all?** So your code depends on the standard, not the vendor — annotate with JPA's `@Entity`, and you could

swap Hibernate for EclipseLink without rewriting entities. Standard interface, replaceable implementation — the exact same philosophy as Hour 1's DI (depend on the interface, swap the implementation)!

## 🗨 INTERVIEW CHECKPOINT — asked in some form in almost every interview

**Q: "Difference between JPA and Hibernate?"** 🐼: "JPA is a specification — annotations and interfaces defining how Java objects map to relational tables, with no implementation. Hibernate is the most popular implementation of that spec — it does the actual SQL generation, caching, and session management. We code against JPA's standard API; Hibernate does the work underneath."

**Q: "Then what does Spring Data JPA add?"** 🐼: "A repository abstraction on top of JPA: I declare an interface extending `JpaRepository` and Spring generates the implementation at runtime — CRUD, pagination, and derived queries from method names — eliminating boilerplate `EntityManager` code."

## 5.3 Entities — teaching the interpreter your vocabulary

The ORM needs a dictionary: which class ↔ which table, which field ↔ which column. You provide it with annotations:

```
@Entity                                // ① "this class maps to a table"
@Table(name = "users")                  // ② table name (default would be the class name)
public class User {

    @Id                                  // ③ "this field is the PRIMARY KEY"
    @GeneratedValue(strategy = GenerationType.IDENTITY) // ④ DB auto-generates it
    private Long id;

    @Column(name = "name", nullable = false, length = 100) // ⑤ column details
    private String name;

    @Column(name = "email", unique = true, nullable = false)
    private String email;

    // JPA REQUIRES a no-arg constructor + getters/setters
}
```

Line by line:

① **@Entity** — the big switch: "Hibernate, manage this class; each object = one row." ② **@Table** — optional rename. Why "users" not "user"? `USER` is a reserved word in many databases — a classic real-world gotcha. ③ **@Id** — every entity MUST have a primary key. No `@Id` → app fails at startup. ④ **@GeneratedValue(IDENTITY)** — "don't make me invent IDs; the database's auto-increment column assigns them on insert." (Other strategies exist — SEQUENCE, UUID — just know `IDENTITY` = DB auto-increment.) ⑤ **@Column** — constraints and details. `nullable=false` → NOT NULL in the table; `unique=true` → unique index. Skip `@Column` entirely and the field still maps, using its own name.

And remember Hour 4's config line — now it means something:

```
spring.jpa.hibernate.ddl-auto=update    # Hibernate creates/updates tables FROM your entities
spring.jpa.show-sql=true                 # print the generated SQL – watch the interpreter work!
```

(`update` is great for learning; real production teams use migration tools and set this to `validate` — knowing that distinction is a nice senior flex.)

## 🗨 INTERVIEW CHECKPOINT

**Q: "What makes a class a JPA entity?"** → 🐼: "@Entity plus a field annotated @Id — a no-arg constructor is required too. @Table/@Column customize names and constraints; defaults apply otherwise." **Q: "What does @GeneratedValue do?"** → 🐼: "Delegates primary-key generation — with `IDENTITY`, the database's auto-increment assigns the id at insert time."

## 5.4 Repositories — declare an interface, get a DAO for free

Now the part that feels like magic the first time. To get full CRUD for `User`, you write... this:

```
public interface UserRepository extends JpaRepository<User, Long> {
}
//           ↑      ↑
//           entity type of its @Id
```

**An empty interface.** No implementation class anywhere. Yet you can immediately:

```

userRepository.save(user);           // INSERT or UPDATE
userRepository.findById(1L);         // SELECT ... WHERE id=1 → Optional<User>
userRepository.findAll();             // SELECT *
userRepository.deleteById(1L);       // DELETE
userRepository.count();               // SELECT COUNT(*)

```

**Where's the implementation?!** At startup, Spring Data JPA sees your interface and **generates an implementing class at runtime** (a proxy — conceptually the same trick as Hour 4's @Configuration proxy). That generated class calls JPA's EntityManager, which Hibernate implements, which emits SQL. You wrote zero lines of it.

The hierarchy from your handbook (page 3) — each level adds ability:

```

Repository<T, ID>           → empty marker ("I'm a repository")
  ↓
CrudRepository<T, ID>       → save, findById, findAll, delete...
  ↓
PagingAndSortingRepository<T, ID> → + findAll(Pageable), findAll(Sort)
  ↓
JpaRepository<T, ID>        → + JPA extras: flush(), saveAndFlush(), batch deletes

```

In practice: **just extend JpaRepository** — it includes everything above it. 🐘 "JpaRepository extends PagingAndSorting extends Crud; extending JpaRepository gives CRUD plus pagination plus JPA-specific operations in one interface."

Why pagination lives here matters: `findAll()` on a million-row table loads a million objects into memory — death. `findAll(PageRequest.of(0, 20))` fetches 20. Pin this — it returns in Hour 6 as one of your LinkedIn scenario questions.

## 5.5 Derived queries — Spring writes SQL from your method NAME

Add methods to the interface — still no implementation — and name them in Spring's grammar:

```

public interface UserRepository extends JpaRepository<User, Long> {

    List<User> findByName(String name);
    //   WHERE name = ?

    User findByEmail(String email);
    //   WHERE email = ?

    List<User> findByNameContaining(String keyword);
    //   WHERE name LIKE %keyword%

    List<User> findByAgeGreaterThanOrderByAgeAsc(int age);
    //   WHERE age > ? ORDER BY age ASC

    boolean existsByEmail(String email);
    long countByCity(String city);
}

```

At startup, Spring **parses the method name like a sentence**: `findBy` (SELECT) + `Name` (must match an entity FIELD) + `Containing` (LIKE) → it builds the query. Grammar words: `And`, `Or`, `Between`, `LessThan`, `GreaterThan`, `Like`, `Containing`, `In`, `OrderBy...Asc/Desc`, `IsNull`, prefixes `existsBy` / `countBy` / `deleteBy`.

The catch that makes this a great interview answer: the name must reference **entity field names, exactly**. `findByFullName` when the field is `name` → **startup failure** ("No property fullName found"). This is a feature: your query-typos explode at boot, not silently at 3 AM.

**When names get too long or logic too complex → @Query:**

```

@Query("SELECT u FROM User u WHERE u.email LIKE %:domain AND u.age > :age")
List<User> findByEmailDomainAndMinAge(@Param("domain") String domain, @Param("age") int age);

```

Note that's **JPQL, not SQL**: it queries the **entity and its fields** (`User u`, `u.email`), not the table and columns. Hibernate translates JPQL → your database's SQL dialect. (Real raw SQL is possible with `nativeQuery = true` — mention it exists.)

### 💬 INTERVIEW CHECKPOINT — guaranteed territory

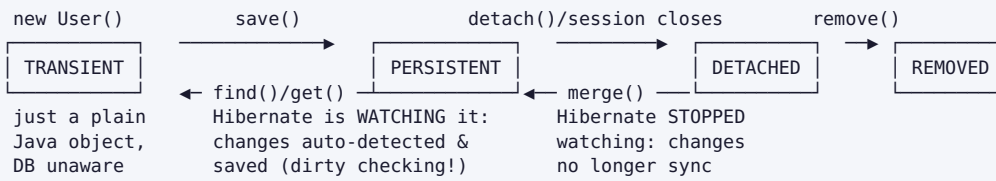
**Q: "How does findByEmail work with no implementation anywhere?"** 🐘 "Spring Data parses the method name against the entity's fields at startup and generates the query and the implementing proxy at runtime. Wrong field names fail fast at boot."

**Q: "JPQL vs SQL?"** 🐘 "JPQL queries entities and their fields; SQL queries tables and columns. Hibernate translates JPQL into the database's SQL dialect, keeping queries database-independent."

**Q: "Derived query methods vs @Query — when which?"** 🐘 "Derived names for simple lookups — self-documenting and validated at startup. @Query when the name would get absurd or the logic needs joins/aggregation. Native queries only when I truly need

## 5.6 Entity states — the lifecycle that explains Hour 6's traps

Last concept, and it quietly explains everything in Hour 6. An entity object is always in one of four states, based on one question: *is Hibernate currently watching it?*



- **Transient** — `new User()`. A plain object; the DB has never heard of it.
- **Persistent** — after `save()` or when loaded via `findById()`. Hibernate is now **actively tracking** it inside its session. The magic consequence — **dirty checking**: modify a persistent entity's field inside a transaction and Hibernate auto-generates the UPDATE at commit. *You often don't even call `save()` for updates.*
- **Detached** — the session closed (e.g., the request ended). The object still exists in Java, but Hibernate stopped watching. Changes go nowhere unless you `merge()` it back.
- **Removed** — marked for deletion.

*Analogy: a patient and a hospital. Transient = a person on the street (hospital has no file). Persistent = admitted patient — nurses monitor continuously; any change in condition is automatically recorded in the file (dirty checking). Detached = discharged — the person still exists, but changes at home update no hospital file, unless readmitted (merge). Removed = file marked for shredding.*

**Why this matters:** Hour 6's famous `LazyInitializationException` — one of your LinkedIn scenario questions — is *literally* "you tried to use a DETACHED entity's lazy data after discharge." Learn these four states now and that bug becomes obvious instead of mysterious.

### 🗣️ INTERVIEW CHECKPOINT

**Q: "Entity states in JPA?"** → 🧠: "Transient — new object, unknown to the DB. Persistent — managed by the session; dirty checking auto-syncs changes. Detached — session closed, no longer tracked; `merge()` reattaches. Removed — scheduled for deletion." **Q: "Why did my entity update without calling `save()`?"** ← the trick version → 🧠: "Dirty checking — it was persistent inside a transaction, so Hibernate flushed the change at commit automatically."

## 5.7 The 30-second story for this hour

"JPA is the specification, Hibernate the implementation doing the actual ORM, and Spring Data JPA the repository layer on top. I annotate classes with `@Entity/@Id` to map them to tables, extend `JpaRepository` to get CRUD and pagination generated at runtime, use derived method names for simple queries — validated at startup — and `@Query` with JPQL for complex ones. Entities move through transient, persistent, detached, and removed states; while persistent, dirty checking syncs changes automatically."

### 🔧 HOUR 5 DRILL — out loud

1. What problem does ORM solve? (object↔table translation; interpreter analogy)
2. JPA vs Hibernate vs Spring Data JPA? (syllabus / driving school / concierge)
3. Why code against a specification? (swap implementations — same philosophy as DI)
4. Minimum for a class to be an entity? (`@Entity` + `@Id` + no-arg constructor)
5. What does `@GeneratedValue(IDENTITY)` mean? (DB auto-increment assigns the id)
6. `CrudRepository` vs `JpaRepository`? (Jpa = Crud + paging/sorting + JPA extras — extend Jpa)
7. How does `findByNameContaining` work with no code? (name parsed against entity fields at startup → generated proxy; wrong name = boot failure)
8. JPQL vs SQL? (entities/fields vs tables/columns; dialect-independent)
9. Four entity states? (transient / persistent / detached / removed — hospital patient)
10. What is dirty checking? (persistent entities auto-UPDATE at commit — no `save()` call needed)

**Next → Hour 6:** Relationships & the Famous JPA Traps — `@OneToMany/@ManyToOne`, LAZY vs EAGER, and your three LinkedIn scenario

questions: LazyInitializationException in production, the N+1 problem at a million rows, and concurrent updates silently overwriting each other.